

# **ROBÓTICA**

## **INFORME CAPÍTULO 6 parte 2 - RVC**

**Localización y mapeo simultáneo. (SLAM)**

**Docente: Jaime Enrique Arango Castro**

**Estudiante:**

Cristhian Mateo Almeida Gómez

**Universidad Nacional de Colombia**

**Ingeniería Electrónica**

**Sede Manizales**



Manizales, Colombia

Julio 2025

# 1. Resumen

El problema de localización y mapeo simultáneo (SLAM) aborda el desafío de determinar la posición del robot mientras construye un mapa del entorno. Esta tarea es análoga a la cartografía marina histórica, donde los navegantes determinaban la ubicación de su embarcación mientras trazaban mapas, como se muestra en la Figura 20.

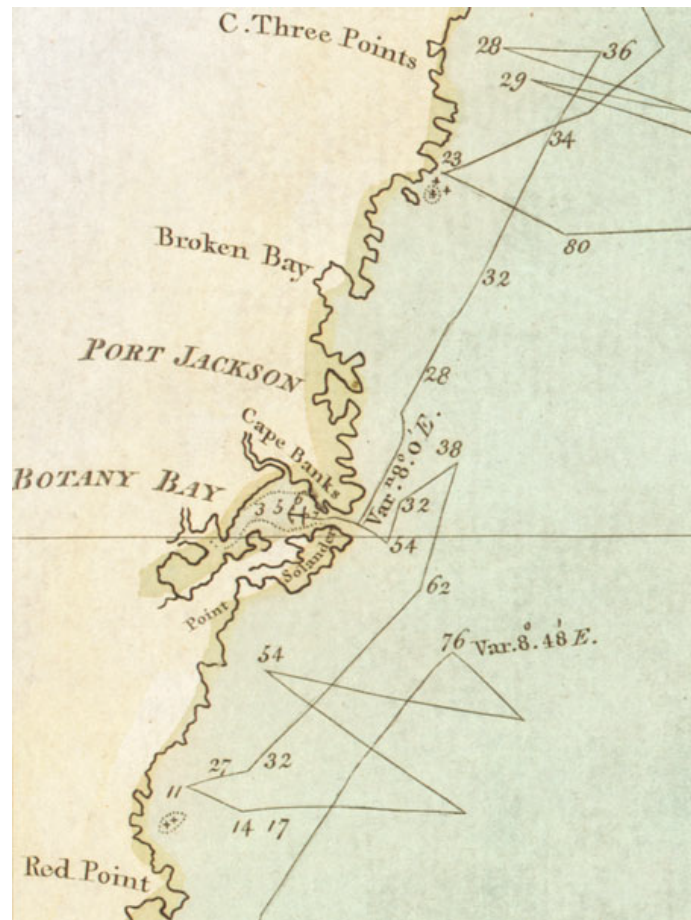


Figura 1: Mapa de la costa oriental de Australia realizado por el capitán James Cook en 1770. La posición del barco y el mapa fueron determinados simultáneamente. (Fuente: Corke, 2023)

En robótica móvil, este problema se denomina SLAM (Simultaneous Localization and Mapping), y consiste en construir un mapa del entorno mientras se estima la localización del robot sin contar con información externa como GPS. El problema se considera una paradoja tipo “el huevo y la gallina”, ya que para ubicarse se requiere un mapa y para hacer el mapa se necesita conocer la posición.

## 2. 6.4 Localización y mapeo simultáneo (SLAM)

### 2.1. Vector de estado y covarianza

El vector de estado  $\mathbf{x} \in R^{(2M+3) \times 1}$  está compuesto por la configuración del robot y las posiciones de los  $M$  landmarks observados hasta el momento:

$$\mathbf{x} = (x_v, y_v, \theta_v, x_0, y_0, x_1, y_1, \dots, x_{M-1}, y_{M-1})^T \quad (1)$$

La covarianza asociada a este estado es una matriz simétrica definida como:

$$\hat{\mathbf{P}} = \begin{bmatrix} \hat{\mathbf{P}}_{vv} & \hat{\mathbf{P}}_{vm} \\ \hat{\mathbf{P}}_{mv} & \hat{\mathbf{P}}_{mm} \end{bmatrix} \in R^{(2M+3) \times (2M+3)} \quad (2)$$

- $\hat{\mathbf{P}}_{vv}$ : Covarianza de la pose del robot.
- $\hat{\mathbf{P}}_{mm}$ : Covarianza de las posiciones de los landmarks.
- $\hat{\mathbf{P}}_{vm}$ : Correlaciones cruzadas entre el robot y los landmarks.

### 2.2. Jacobiana de inserción

Cuando se detecta un nuevo landmark, se debe insertar en el vector de estado. Para ello, se usa la jacobiana de inserción  $G_x$ , que modela cómo afecta la posición del landmark a la configuración actual del robot:

$$G_x = \frac{\partial g}{\partial \mathbf{x}} = \begin{bmatrix} 1 & 0 & -r \sin(\theta_v + \beta) \\ 0 & 1 & r \cos(\theta_v + \beta) \end{bmatrix} \quad (3)$$

donde:

- $r$ : distancia medida al landmark.
- $\beta$ : ángulo de observación respecto a la orientación del robot.
- $\theta_v$ : orientación del robot.

### 2.3. Jacobiana de observación

Para actualizar la estimación con observaciones, se emplea la jacobiana  $H_x$ , la cual relaciona el cambio en la observación con el cambio en el vector de estado. Se construye tomando en cuenta solo los elementos que afectan directamente la observación (robot y landmark observado):

$$H_x = [H_{x_v} \quad \mathbf{0}_{2 \times 2} \quad \dots \quad H_{x_i} \mathbf{0}_{2 \times 2} \quad \dots] \quad (4)$$

—

## 2.4. Implementación en Robotics Toolbox

La Toolbox implementa SLAM utilizando un modelo de observación de rango y orientación. A continuación, se define un mapa con landmarks aleatorios, se configura un robot tipo bicicleta, y se ejecuta el filtro EKF, además de definir una matriz de covarianza de error inicial, con el fin de representar la incertidumbre inicial en la pose del robot y los landmarks:

```
map = LandmarkMap(20, workspace=10);
W = np.diag([0.1, np.deg2rad(1)]) ** 2
robot = Bicycle(covar=V, x0=(3, 6, np.deg2rad(-45)),
               animation="car");
robot.control = RandomPath(workspace=map);
W = np.diag([0.1, np.deg2rad(1)]) ** 2
sensor = RangeBearingSensor(robot=robot, map=map, covar=W,
                             range=4, angle=[-pi/2, pi/2]);
P0 = np.diag([0.05, 0.05, np.deg2rad(0.5)]) ** 2;
ekf = EKF(robot=(robot, V), P0=P0, sensor=(sensor, W));
✓ 1.0s
```

Figure 1

Figura 2: implementacion en robotics toolbox

Este código genera un entorno simulado donde el robot parte de una pose conocida  $(3, 6, -45\text{grados})$  y recorre un mapa con 20 landmarks aleatorios. El filtro EKF estima tanto su localización como la de los landmarks a lo largo del tiempo, manejando la incertidumbre mediante la matriz de covarianza  $\hat{\mathbf{P}}$ .

## 2.5. Algoritmo EKF-SLAM

1. **Predicción:** Usa el modelo de movimiento para predecir el nuevo estado y propagar incertidumbre.
2. **Actualización:** Integra observaciones (rangos, ángulos) mediante el filtro de Kalman extendido (EKF).
3. **Inserción de landmarks:** Se actualiza el estado y la covarianza cuando se detectan nuevos puntos.

```
# ekf.run(T=40);
ekf.run_animation(T=40)

#html = ekf.run_animation(T=40, format="html")
#HTML(html)
[6] ✓ 0.7s
```

<matplotlib.animation.FuncAnimation at 0x1d96fa5a450>

Figure 2

Figura 3: implementacion algoritmo EKF en robotics toolbox

El sensor de rango y ángulo (RangeBearingSensor) se utiliza para medir la distancia y el ángulo de los landmarks con respecto al robot, lo que ayuda a generar las observaciones necesarias para el filtro de Kalman. El EKF se inicializa con una matriz de covarianza de error inicial,  $P_0$ , que representa la incertidumbre inicial en la pose del robot y los landmarks. Luego, el filtro EKF se ejecuta durante 40 segundos para obtener las estimaciones de la trayectoria del robot y la localización de los landmarks.

Para comenzar, podemos imprimir la trayectoria real seguida por el robot en nuestro mapa de 20 puntos de referencia.

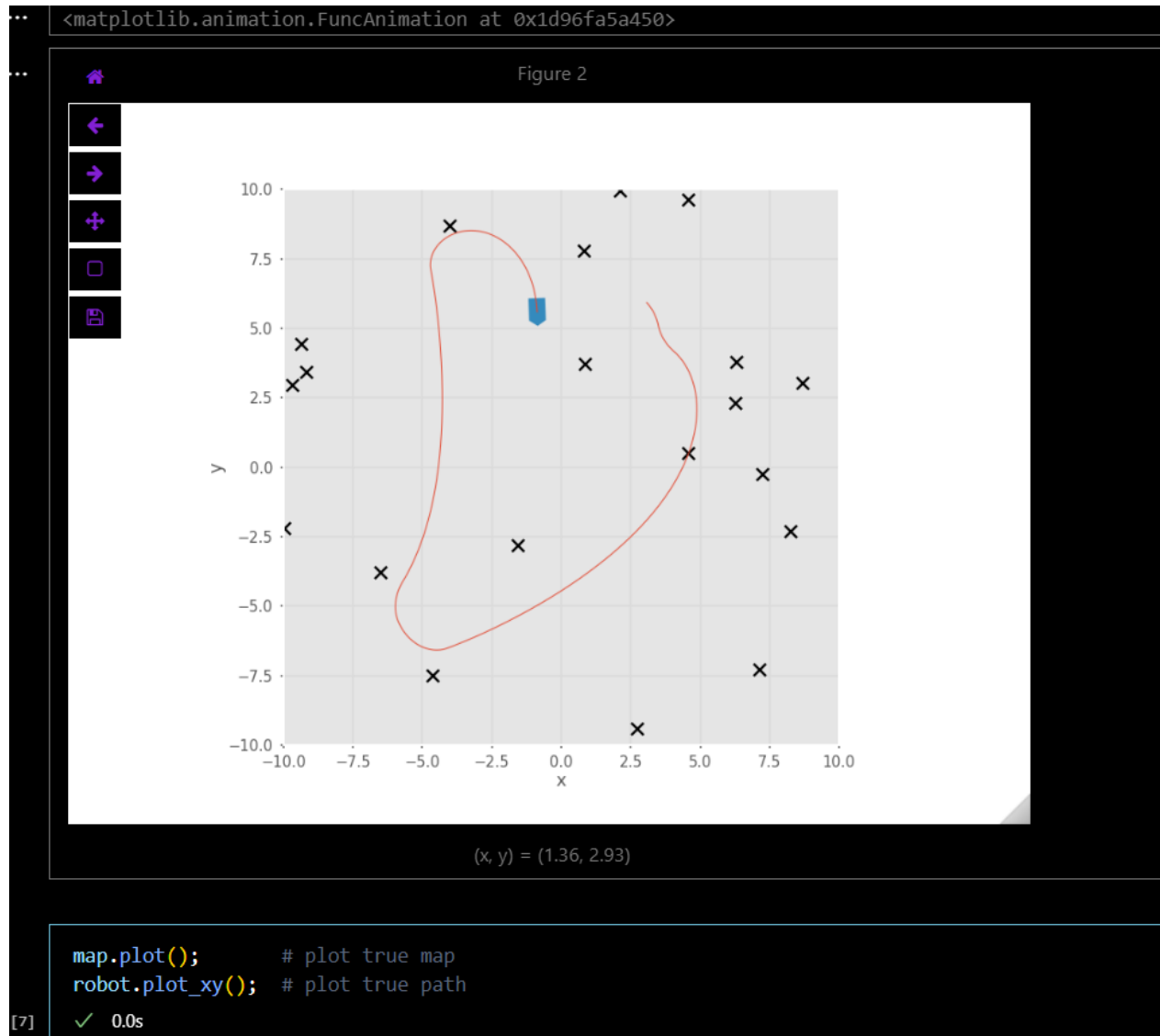


Figura 4: ruta real

Una vez ejecutado el filtro durante 40 segundos, se visualizan múltiples aspectos del desempeño del sistema SLAM: las trayectorias, posiciones estimadas de los landmarks, la incertidumbre del sistema y las correlaciones entre estados.

La Figura 4 muestra una comparación entre la trayectoria real del robot (izquierda) y la estimación producida por el filtro (derecha), junto con la localización estimada de los landmarks y las elipses de incertidumbre al 95 % de confianza. Posteriormente, podemos usar métodos del algoritmo ekf, los cuales ayuden a interpretar el funcionamiento de este :

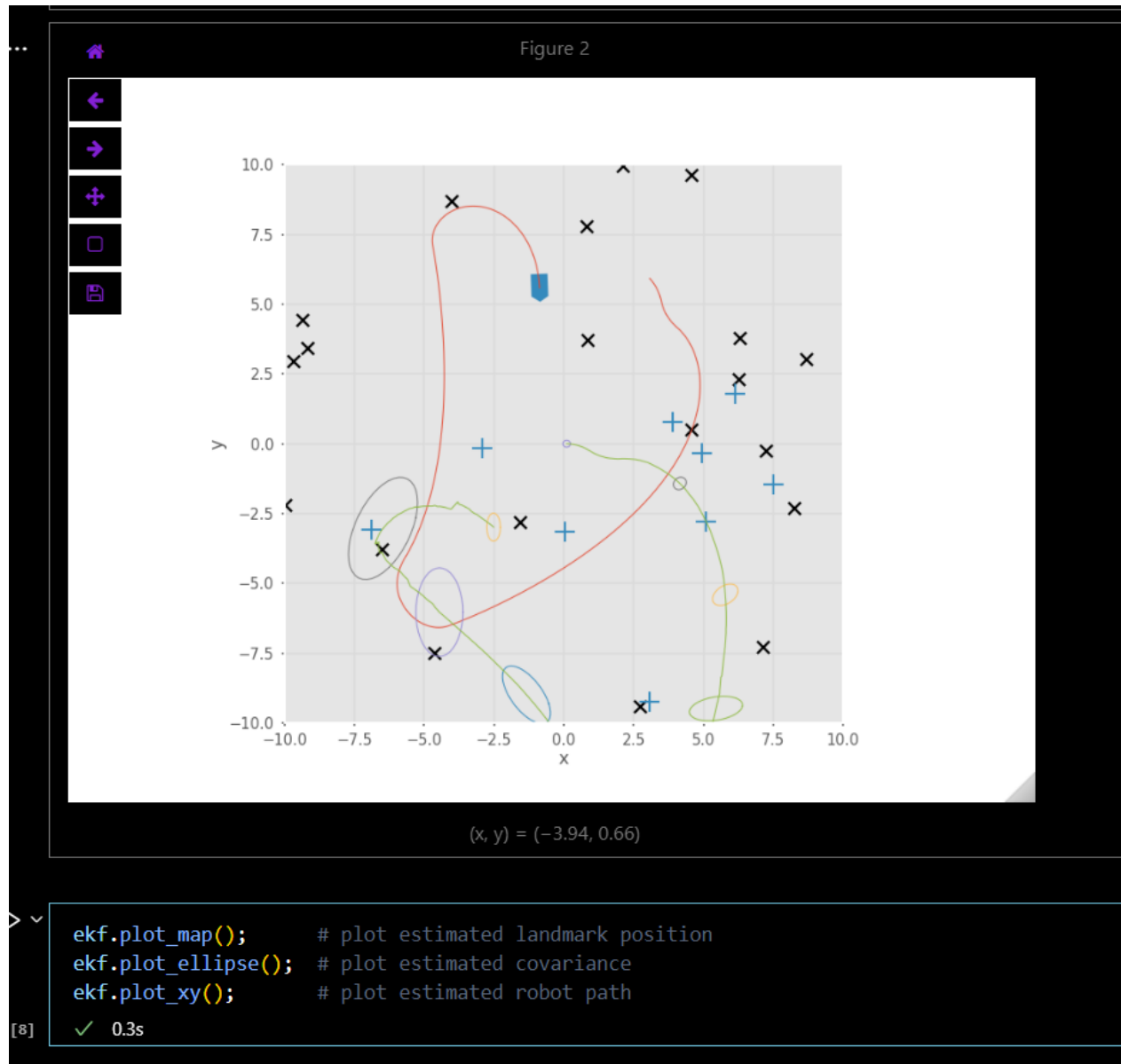


Figura 5: Uso métodos del ekf

Vemos como estos métodos, permiten visualizar las estimaciones del mapa, la incertidumbre del estimador (como elipses de confianza), y la trayectoria estimada del robot. Las elipses de incertidumbre visualizan el grado de confianza en las estimaciones del robot y los landmarks, con un mayor tamaño de la elipse indicando una mayor incertidumbre.

**Observación:** La diferencia significativa entre la trayectoria real y estimada se debe a que el filtro EKF parte de una suposición errónea de configuración inicial:  $(0, 0, 0)$  en lugar de  $(3, 6, -45^\circ)$ . Esta diferencia puede corregirse aplicando una transformación  $SE(2)$  para alinear el marco estimado con el real:

```
1 T = ekf.get_transform(map)
```

Esta transformación rota y traslada los estados estimados al marco del mundo, mostrando un alineamiento más preciso entre los caminos reales y estimados.

## 2.6. Transformación entre marcos y estructura de la matriz de covarianza

Una de las tareas más importantes del SLAM es alinear el mapa estimado con el mundo real. Para esto, se puede obtener una transformación  $SE(2)$  que permite transformar la trayectoria y landmarks estimados al marco global:

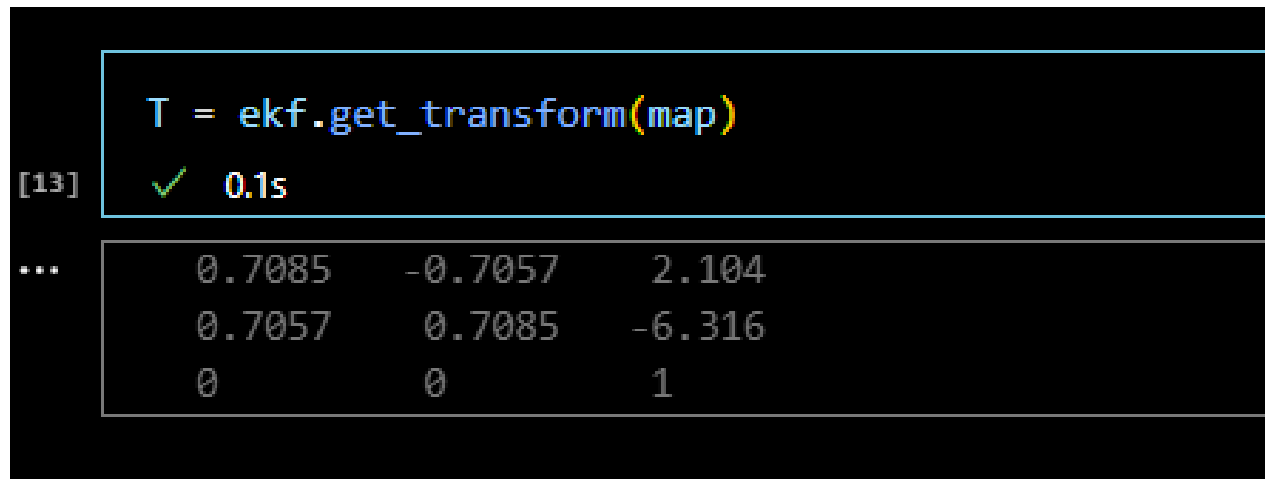


Figura 6: Transformación entre marcos.]

Esta transformación consiste en una matriz homogénea que representa una rotación y traslación:

Con esta transformación, los datos del filtro se alinean con el mapa de referencia, como se observa en la Figura 7. Esto confirma que, pese a una configuración inicial incorrecta, el sistema logra una buena estimación relativa y permite alinearla posteriormente.

## 2.7. Evolución de la incertidumbre

A medida que se observan más landmarks, la traza de la matriz de covarianza  $P$  disminuye. Además, se introducen correlaciones entre landmarks, haciendo que  $P$  deje de ser bloque-diagonal. Esto refleja una mejora en la confianza del sistema global.

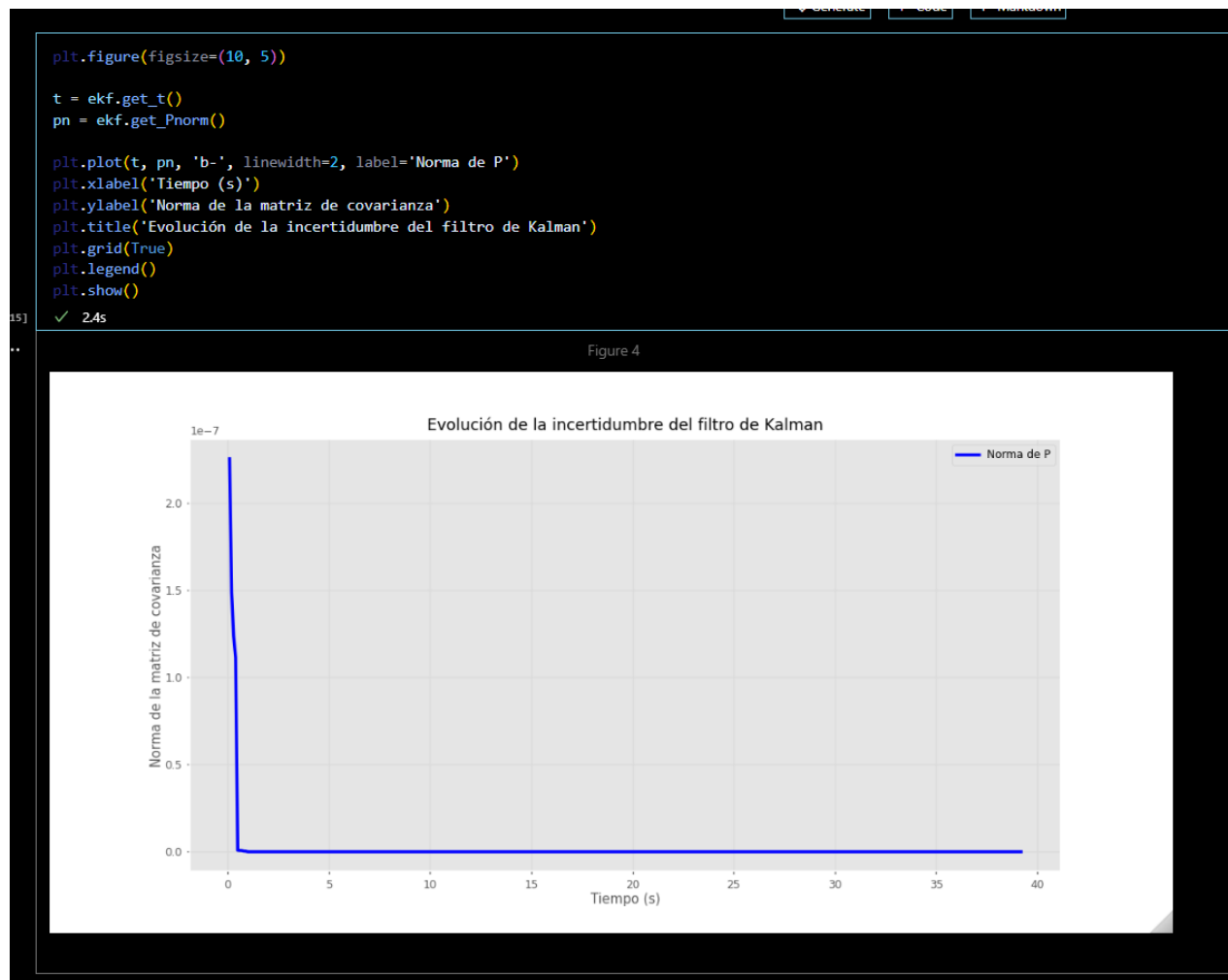


Figura 7: Covarianza en función del tiempo.]

Veamos en escala logarítmica.

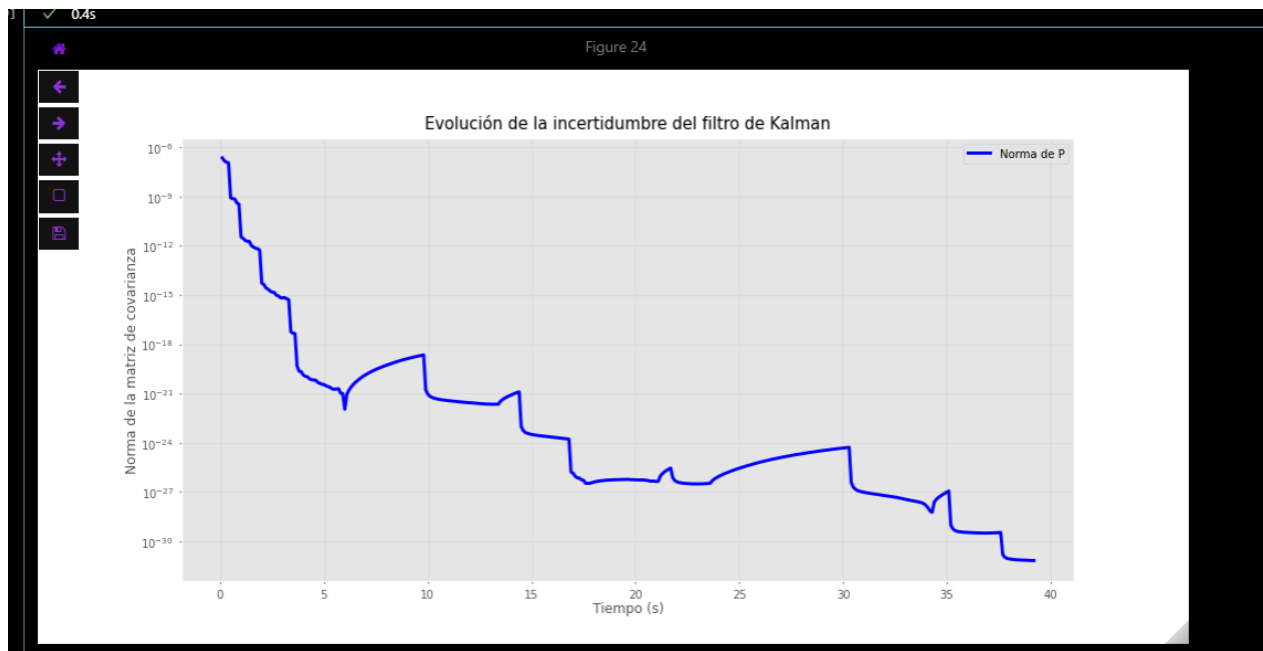


Figura 8: Covarianza en funcion del tiempo en escala logarítmica.]

Tal como vemos en la grafica de la evolución de la incertidumbre, esta muestra cómo disminuye la incertidumbre en la estimación de la posición del robot con el tiempo. A medida que el robot observa más landmarks y obtiene más información, la covarianza disminuye, lo que indica una mayor precisión en la estimación de la localización.

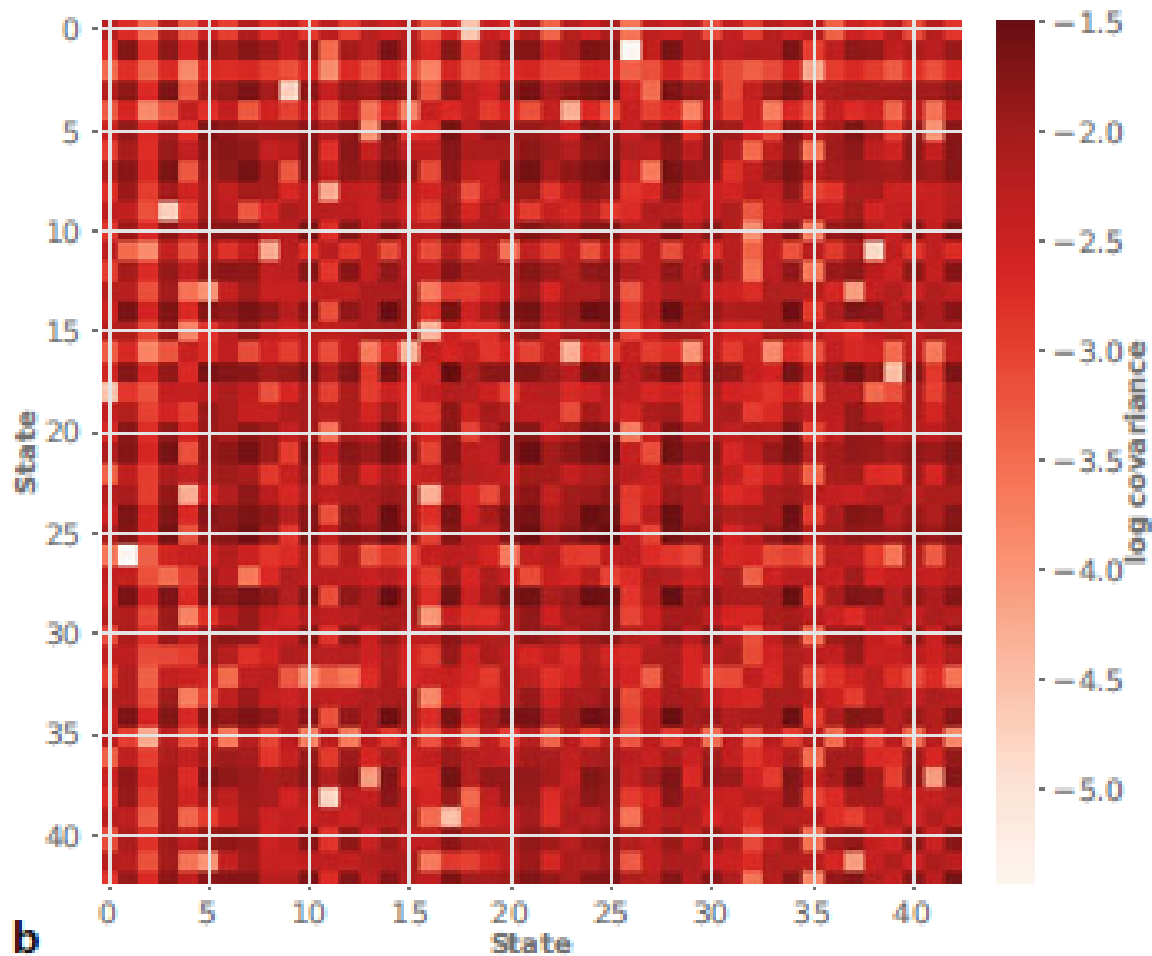


Figura 9: Matriz de covarianza final]

De igual forma, tenemos la gráfica para el matriz de covarianza final, lo que permite observar cómo se correlacionan los errores en la posición del robot con los errores en la localización de los landmarks. Las celdas fuera de la diagonal indican correlación entre la pose del robot y las posiciones de los landmarks.

La estructura de la matriz refleja las correlaciones entre los diferentes estados del sistema, lo que es clave para la eficiencia del filtro de Kalman.

**Interpretación conceptual acerca del algoritmo.** El comportamiento del EKF puede interpretarse como una red de resortes: cualquier error en la estimación de un estado (robot o landmark) se propaga a los demás debido a las conexiones implícitas entre ellos.

## 2.8. Limitaciones del EKF en SLAM

Aunque el EKF es una herramienta poderosa, tiene restricciones prácticas importantes:

- El tamaño de la matriz de covarianza crece cuadráticamente con el número de landmarks, lo que afecta rendimiento y escalabilidad.
- El filtro asume distribución Gaussiana del ruido, lo cual no es siempre válido para sensores reales como LIDAR o IMU.
- Requiere una buena estimación de la covarianza del ruido, que en entornos reales es difícil de conocer con precisión.

### 3. 6.8 Aplicación: LIDAR

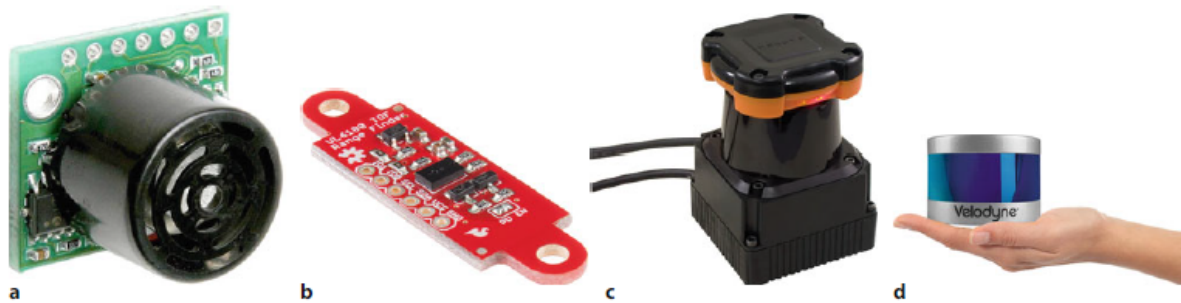


Figura 10: Ejemplos de sensores

El LIDAR (Light Detection and Ranging) es un sensor crítico en SLAM, ya que permite realizar mapeo y localización precisos en entornos complejos. El principio básico del LIDAR es que emite pulsos de luz láser y mide el tiempo que tarda en regresar, proporcionando una medida de distancia precisa. Esta tecnología es fundamental para los sistemas robóticos autónomos, permitiéndoles generar un mapa detallado de su entorno y estimar su posición en tiempo real.

Existen diversas configuraciones de LIDAR utilizadas en aplicaciones de robótica móvil, desde sensores de un solo rayo hasta versiones más avanzadas que proporcionan escaneos en 3D, lo cual mejora considerablemente la precisión y la calidad de los mapas generados.

#### 3.1. 6.8.1 Principio de funcionamiento

El funcionamiento del LIDAR se basa en la emisión de pulsos de luz láser que se dirigen hacia el entorno del robot. Cuando el pulso golpea un objeto, una parte de la luz se refleja de vuelta hacia el sensor, que mide el tiempo que tarda en regresar. Utilizando esta información, el sensor puede calcular la distancia a los objetos cercanos.

Al girar el sensor, este proceso se repite en diferentes ángulos, lo que permite al LIDAR crear una representación tridimensional del entorno. Estos datos se utilizan para estimar la posición del robot y construir un mapa de los objetos y obstáculos en su entorno.

El LIDAR es extremadamente útil porque puede generar datos precisos incluso en condiciones de baja iluminación, lo que lo hace ideal para su uso en la oscuridad, a diferencia de cámaras u otros sensores que requieren luz para obtener imágenes claras.

### 3.2. 6.8.2 Tipos de LIDAR

Existen diferentes tipos de LIDAR, cada uno adecuado para tareas específicas en el ámbito de la robótica móvil y la localización. Los tipos más comunes son:

- **1D-LIDAR:** Utiliza un solo rayo láser para medir la distancia a los objetos en una dirección. Es adecuado para aplicaciones simples donde no se requiere un mapeo detallado del entorno. Un ejemplo típico de un sensor 1D-LIDAR es el sensor utilizado en algunos robots móviles básicos.
- **2D-LIDAR:** En este tipo de LIDAR, el sensor emite un rayo láser que rota en un plano, generando un escaneo bidimensional del entorno. Este tipo de LIDAR es muy utilizado en aplicaciones de mapeo y navegación en entornos planos. Los robots móviles autónomos utilizan a menudo LIDAR 2D para navegar por interiores o en espacios donde no hay grandes variaciones en la altura de los objetos.
- **3D-LIDAR:** Los sensores 3D-LIDAR tienen la capacidad de emitir múltiples rayos en diferentes ángulos y alturas, lo que permite crear un mapa tridimensional del entorno. Esto proporciona una visión más completa y precisa del entorno y es útil para aplicaciones complejas de robótica móvil en entornos dinámicos y no estructurados. Los 3D-LIDARs son utilizados frecuentemente en vehículos autónomos y otros sistemas robóticos avanzados que necesitan captar una representación detallada en 3D de su entorno.

#### Mapping

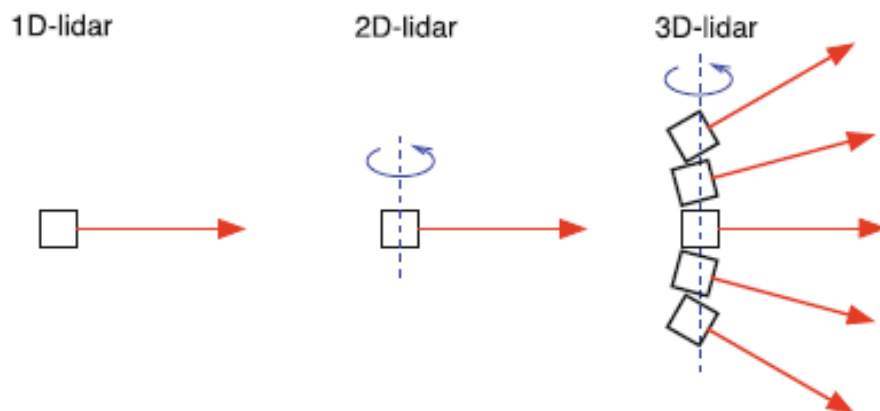


Figura 11: Tipos de lidarY]

Cada tipo de LIDAR tiene sus ventajas y limitaciones. Los 1D y 2D son más económicos y adecuados

para tareas simples, mientras que los 3D son más costosos pero ofrecen una mayor precisión y resolución espacial, siendo ideales para mapeo y navegación en entornos complejos.

### 3.3. 6.8.3 Ventajas y desventajas

El uso del LIDAR en aplicaciones de SLAM presenta varias ventajas y desventajas. A continuación, se describen los aspectos más relevantes:

#### Ventajas

- **Alta precisión y resolución espacial:** El LIDAR ofrece una alta precisión en la medición de distancias, lo que permite generar mapas detallados y precisos del entorno.
- **Capacidad de operar en oscuridad:** A diferencia de las cámaras, los sensores LIDAR no requieren luz ambiental para funcionar, lo que les permite operar en condiciones de baja luminosidad o incluso en completa oscuridad.
- **Escaneo en tiempo real:** Los sensores LIDAR pueden realizar escaneos a alta velocidad, lo que permite a los robots actualizar su mapa y su localización en tiempo real, lo que es crucial para aplicaciones de navegación autónoma.

#### Desventajas

- **Sensible a la reflectividad y a la luz solar intensa:** El LIDAR puede verse afectado por la reflectividad de las superficies que escanea y la presencia de luz solar intensa, lo que puede interferir con la medición precisa de las distancias.
- **Mayor consumo energético:** Los sensores LIDAR requieren más energía en comparación con otros sensores como las cámaras o los ultrasonidos, lo que puede ser una limitación en sistemas robóticos que dependen de fuentes de energía limitadas.
- **No ideal para robots muy rápidos:** El tiempo de escaneo de los sensores LIDAR puede ser problemático para robots de alta velocidad, ya que puede ser difícil obtener datos precisos cuando el sensor no tiene tiempo suficiente para escanear el entorno antes de moverse nuevamente.

A pesar de estas desventajas, el LIDAR sigue siendo uno de los sensores más utilizados en la robótica autónoma debido a sus ventajas en términos de precisión y capacidad para operar en condiciones difíciles.

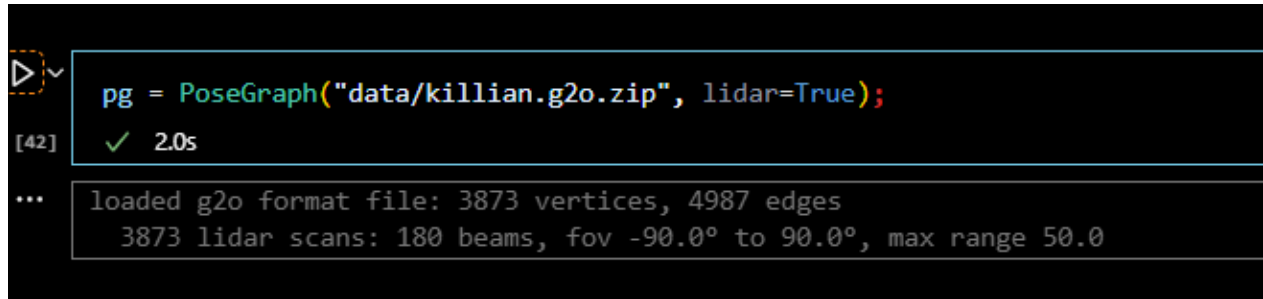
### 3.4. 6.8.1 Odometría basada en LIDAR

Buscamos estimar el cambio de pose del robot usando datos de escaneo Lidar en lugar de la odometría de ruedas.

Una aplicación común de los sensores LIDAR es la estimación de la odometría del robot, es decir, el cambio en la pose del robot utilizando datos de escaneo LIDAR, en lugar de depender de los encoders de rueda. A continuación, se ilustra este proceso utilizando datos reales de un robot móvil.

## Descripción general del código

En el código proporcionado, se utiliza la clase `PoseGraph` para cargar los datos del escaneo LIDAR y calcular el cambio en la pose entre dos ubicaciones consecutivas del robot. El proceso comienza cargando un grafo de pose optimizado y asociando los escaneos de LIDAR con vértices en el grafo.



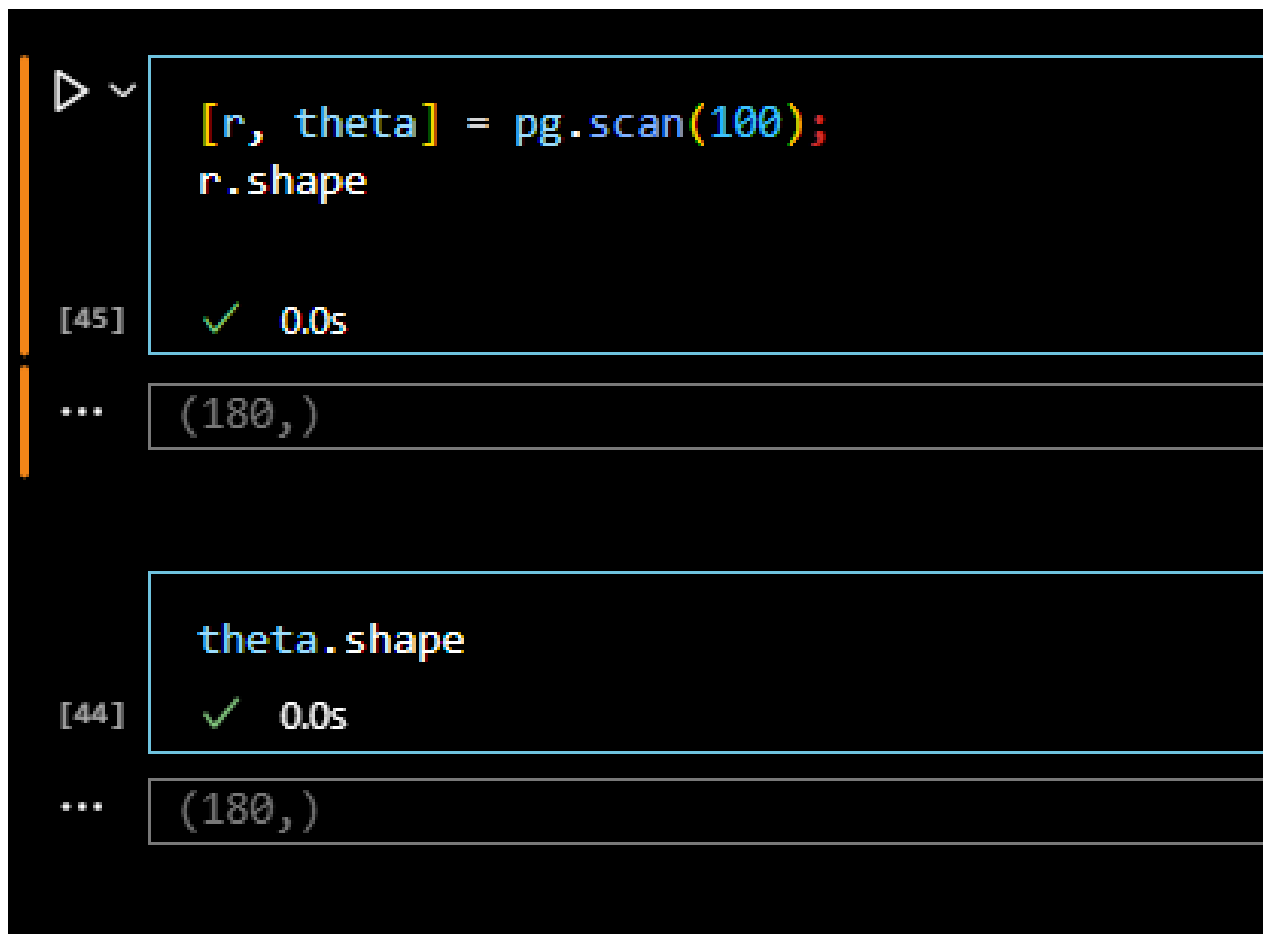
```
▶ [42]: pg = PoseGraph("data/killian.g2o.zip", lidar=True);  
✓ 2.0s  
... loaded g2o format file: 3873 vertices, 4987 edges  
3873 lidar scans: 180 beams, fov -90.0° to 90.0°, max range 50.0
```

Figura 12: Lidar basado en odometría

Aquí, la variable `pg` representa el grafo de poses cargado desde el archivo `killian.g2o.zip`, que contiene tanto los vértices como las aristas optimizadas del grafo, además de los datos del escaneo LIDAR.

## Análisis de los escaneos de LIDAR

A continuación, se obtiene el escaneo de LIDAR de un vértice específico del grafo. En el ejemplo, se extrae el escaneo correspondiente al vértice 100:



```
[r, theta] = pg.scan(100);  
r.shape
```

[45] ✓ 0.0s

... (180,)

```
theta.shape
```

[44] ✓ 0.0s

... (180,)

Figura 13: Análisis escaneo LIDAR

Aquí, `r` y `theta` representan los valores del rango y los ángulos del escaneo LIDAR, respectivamente. El comando `scan(100)` extrae el escaneo del vértice 100, que contiene 180 mediciones de rango (una por cada ángulo en el escaneo).

Luego, se puede graficar el escaneo en coordenadas polares utilizando el siguiente código:

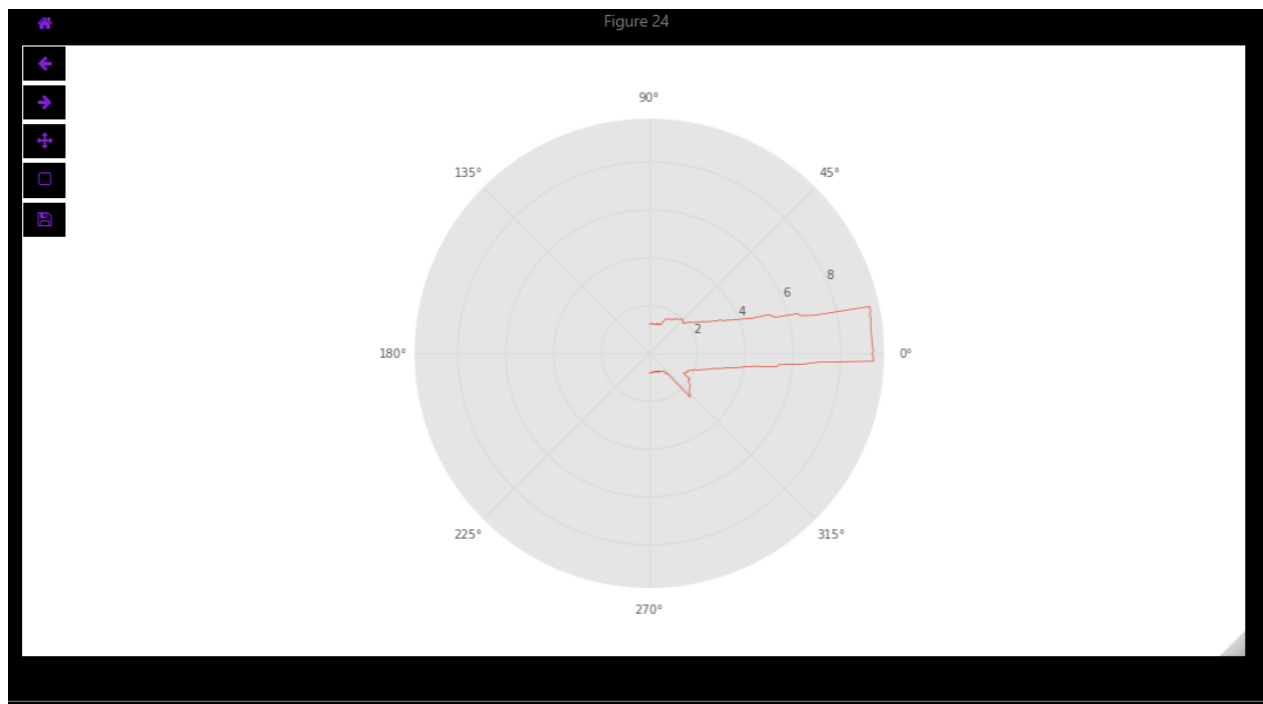


Figura 14: Escaneo en coordenadas polares

Este código crea una representación polar del escaneo LIDAR, donde el eje radial ( $r$ ) muestra la distancia a los objetos, y el ángulo ( $\theta$ ) indica la orientación de la medición.

### Comparación entre dos escaneos consecutivos

Para estimar el cambio en la pose entre dos ubicaciones consecutivas del robot, utilizamos dos escaneos LIDAR consecutivos, por ejemplo, de los vértices 100 y 101:

```
p100 = pg.scanxy(100);
p101 = pg.scanxy(100);
p100.shape
```

[48] ✓ 0.0s

```
...
```

(2, 180)

```
p101.shape
```

[51] ✓ 0.2s

```
...
```

(2, 180)

Figura 15: Escaneos de dos vértices consecutivos

Los dos escaneos consecutivos se almacenan en `pg1` y `pg2`. Estos son matrices de  $180 \times 2$  que contienen las coordenadas  $(x, y)$  de los puntos escaneados en el espacio cartesiano. La comparación de estos puntos permite estimar el cambio en la pose del robot entre las ubicaciones de los vértices 100 y 101.

### Determinación del cambio en la pose del robot

Para calcular el cambio en la pose del robot entre dos escaneos consecutivos, se utiliza un algoritmo de coincidencia de escaneos, como el **scan-matching**, que alinea los dos conjuntos de puntos y calcula la transformación que mejor los ajusta. La transformación se expresa en términos de traslación y rotación.

El código para calcular la transformación relativa entre los dos escaneos es el siguiente:

```
T = pg.scanmatch(100, 101);
T.println()
[49] ✓ 0.2s
...
ITER 0 999997.596411268 2.403588732006146
ITER 1 0.7221130437464967 1.6814756882596493
ITER 2 0.3923791519694475 1.2890965362902018
ITER 3 0.6410190269219513 0.6480775093682505
ITER 4 0.18601299152934603 0.46206451783890445
ITER 5 0.061131785319154364 0.4009327325197501
ITER 6 0.05149040620395645 0.34944232631579364
ITER 7 0.042144810439075475 0.30729751587671816
ITER 8 0.033216275089173686 0.2740812407875445
ITER 9 0.10215952758433464 0.17192171320320984
ITER 10 0.005432907099159279 0.16648880610405056
ITER 11 0.005289990692388141 0.16119881541166242
ITER 12 0.002503931709508661 0.15869488370215376
ITER 13 0.002187693034604782 0.16088257673675854
ITER 14 2.6928311810642258e-05 0.1608556484249479
t = 0.506, -0.00872; 2.93°
```

Figura 16: Transformación de poses

Aquí,  $T$  es la transformación relativa entre los dos escaneos, donde el valor  $[0.506, -0.00872, 2.93]$  indica una traslación de 0.506 metros en la dirección  $x$ , una traslación de -0.00872 metros en  $y$ , y una rotación de 2.93 grados.

Finalmente, calculamos la transformación entre dos escaneos :

```
▶ pg.time(101) - pg.time(100)
[50] ✓ 0.0s
... 1.7599999904632568
```

Figura 17: Calculando la transformación entre dos escaneos

### Visualización de los escaneos

Para visualizar cómo se alinean los puntos de los dos escaneos después de aplicar la transformación, se puede graficar los puntos originales y los transformados:

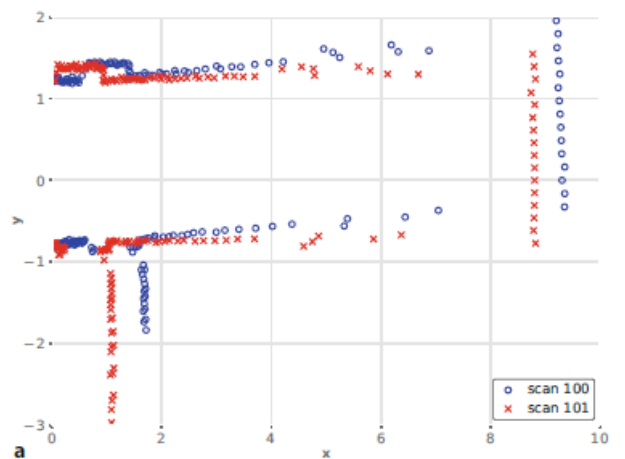


Figura 18: Scaneo laser de la posición 100 y 101

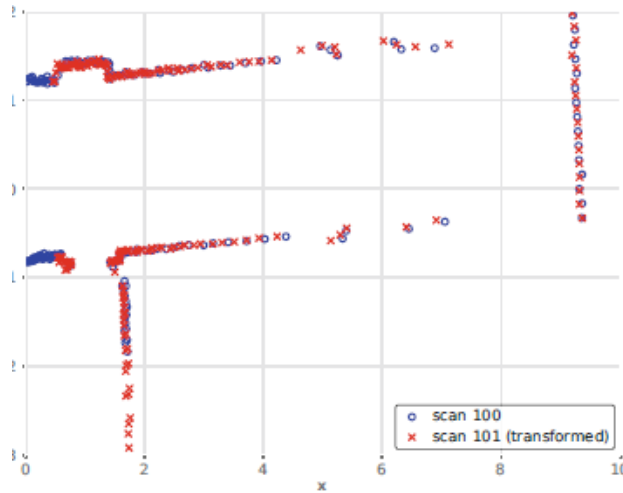


Figura 19: transformación de puntos 100 a 101

En la gráfica se aprecian los escaneos de los vértices 100 y 101, donde los puntos del primer escaneo se muestran en azul, los puntos del segundo escaneo en rojo y la versión transformada del primer escaneo en la siguiente figuraz. Esto permite observar cómo los escaneos se alinean después de la aplicación de la transformación.

### Desafíos en la práctica

En la práctica, existen varios desafíos que pueden afectar la precisión de la odometría basada en LIDAR, como se menciona en el libro:

- **Reflexiones de la superficie:** Los pulsos LIDAR pueden no regresar correctamente si impactan con superficies altamente reflectantes o superficies pulidas que desvían el láser.
- **Sensores en movimiento:** El movimiento del robot puede cambiar temporalmente la forma del entorno y afectar las mediciones de distancia.
- **Condiciones exteriores:** Las precipitaciones o la luz solar intensa pueden interferir con los pulsos LIDAR y reducir la precisión de las mediciones.
- **Ruido en los datos:** El LIDAR, al igual que otros sensores, está sujeto a ruido que puede afectar la calidad de los escaneos.

El algoritmo de *Iterated Closest Point* (ICP) es una técnica estándar para minimizar los errores de alineación y es utilizado en la práctica para mejorar la coincidencia de escaneos.

### 3.5. 6.8.2 Lidar-Based Map Building

Si la pose del robot es lo suficientemente conocida, entonces podemos transformar todos los escaneos de LIDAR a un marco de referencia global y construir un mapa. Existen diversas representaciones de mapas

que podrían utilizarse, pero en esta sección se empleará una cuadrícula de ocupación como se describe en el Capítulo 5.

Para un robot en una pose dada, cada rayo del LIDAR es un vector que nos dice varias cosas. Primero, la distancia medida es la distancia al obstáculo más cercano y podemos determinar las coordenadas de la celda que contiene un obstáculo. Sin embargo, no podemos saber nada acerca de las celdas detrás del rayo: el LIDAR solo detecta el primer objeto frente al sensor.

A medida que el robot se mueve por el entorno, cada celda es interceptada por muchos rayos y usamos un esquema de votación simple para determinar si cada celda está libre o ocupada. Basado en la pose del robot, cada rayo se modela mediante un algoritmo de línea de Bresenham para determinar qué celdas del mapa se deben actualizar. Si el sensor no recibe ningún retorno (por ejemplo, si la superficie está demasiado distante o no es reflectante), omitimos el rayo.

```
og = OccupancyGrid(workspace=[-100, 250, -100, 250], cellsize=0.1, value=np.int32(0));
pg.scanmap(og, maxrange=40)
og.plot(cmap="gray")
3] ✓ 35.1s
```

Figura 20: Construcción del mapa de ocupación usando LIDAR

En el código anterior, creamos un mapa de ocupación utilizando la clase `OccupancyGrid`, que define el área de trabajo y el tamaño de cada celda en la cuadrícula. Luego, los escaneos de LIDAR se transforman en un mapa utilizando la función `pg.scanmap`, y finalmente se genera una visualización del mapa con `og.plot(cmap="gray")`.

El resultado de esta operación es mostrado en la Figura 21 y 22 . En la visualización, las celdas blancas indican espacio libre, las celdas negras indican celdas ocupadas y las celdas grises son desconocidas. El tamaño de las celdas en la cuadrícula es de 10 cm.

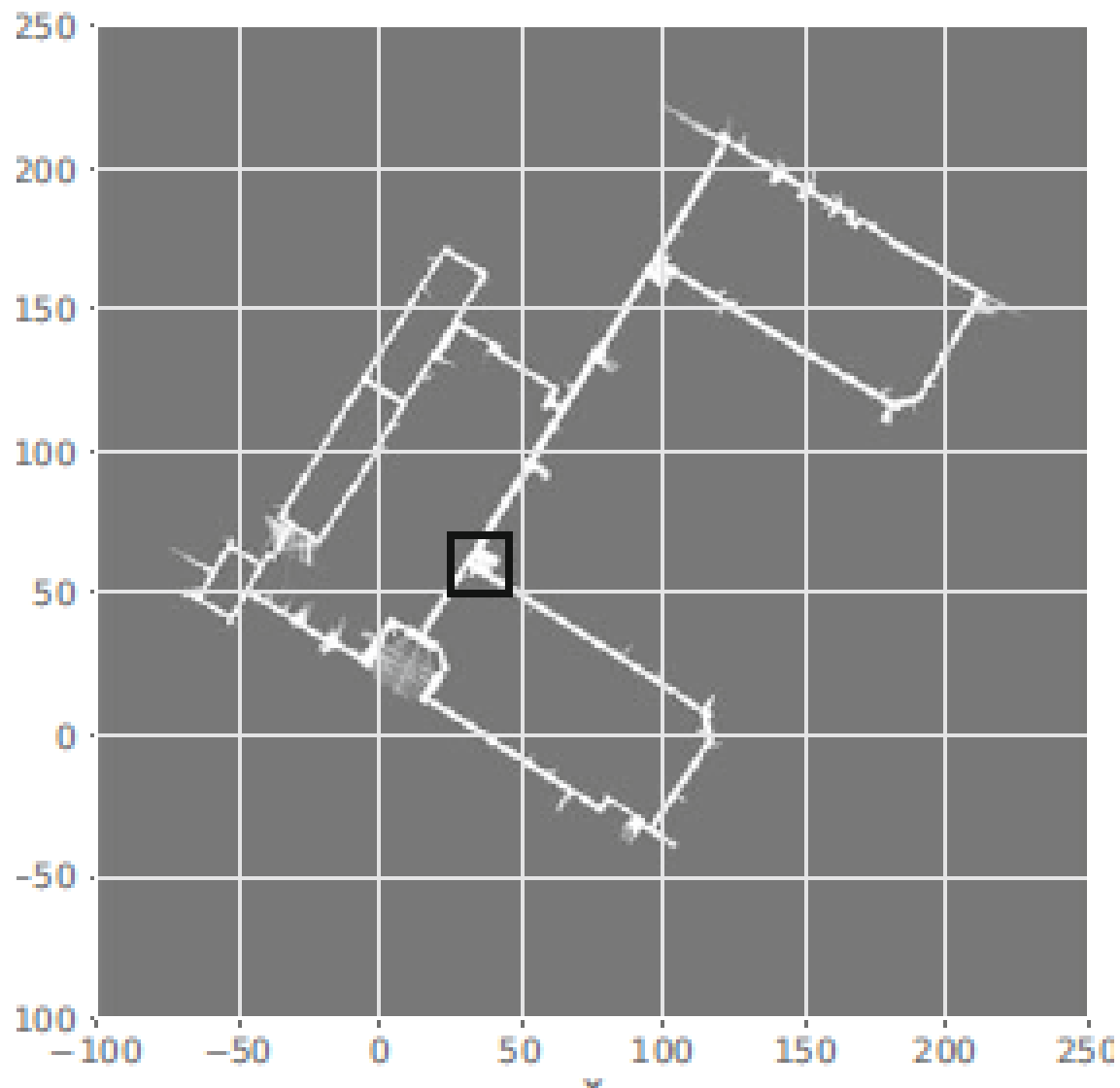


Figura 21: El escaneo de LIDAR representado en una cuadrícula de ocupación.

### 3.6. 6.8.3 Lidar-Based Localization

En esta sección, hemos mencionado varias veces los "landmarks", o puntos de referencia, pero sin proporcionar ejemplos concretos de lo que son. Los "landmarks" pueden ser características distintivas del entorno, como se discutió en la Sección 12.3, o marcadores artificiales, como se discutió en la Sección 13.6.1. Si consideramos un escaneo de LIDAR, como se muestra en la Figura 22, podemos ver un arreglo distintivo de puntos que podemos usar como "landmark".

Los "landmarks" son especialmente útiles cuando el robot navega en entornos que tienen una estructura geométrica clara. En muchos casos, el arreglo de puntos del escaneo LIDAR es ambiguo y de poco valor, por ejemplo, en un pasillo largo donde todos los puntos están alineados, pero algunos de estos puntos pueden ser

altamente únicos y servir como un "landmark" útil.

Nuevamente, podríamos hacer coincidir el escaneo LIDAR actual con todos los escaneos anteriores y si el ajuste es bueno (el error ICP es bajo), podríamos agregar otra restricción al grafo de poses. Sin embargo, este enfoque sería costoso con un gran número de escaneos, por lo que normalmente solo se revisan los escaneos cercanos a la posición estimada del robot.

Sin embargo, el problema de asociación de datos sigue siendo un desafío importante: puede haber varias opciones para emparejar los puntos del escaneo actual con los puntos de referencia anteriores. El algoritmo de "Iterated Closest Point" (ICP) se utiliza para minimizar la distancia entre los puntos correspondientes y mejorar la precisión de la localización.

El código para implementar ICP sería algo como lo siguiente:

```
1 >>> T = icp(scan, previous_scan)
2 >>> pg.add_constraint(T)
```

Listing 1: Aplicando ICP para mejorar la localización

Aquí, el algoritmo ICP se podría aplicar para alinear el escaneo actual con un escaneo anterior, y el resultado de esta alineación se usa para agregar una nueva restricción al grafo de poses. Esto mejora la estimación de la posición del robot al reducir la incertidumbre en la localización.

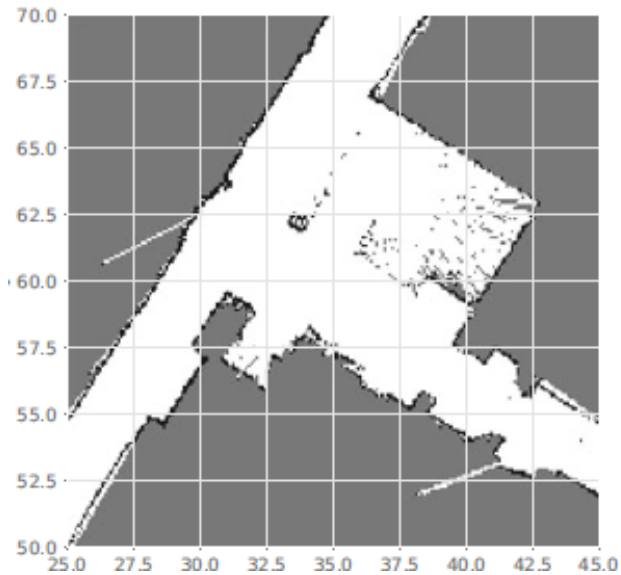


Figura 22: Ejemplo de un escaneo LIDAR utilizado para la localización.

## 4. Conclusiones

- El uso de sensores LIDAR en el proceso de SLAM (Simultaneous Localization and Mapping) es fundamental para lograr una localización precisa en entornos complejos. Los escaneos de LIDAR permiten generar mapas detallados del entorno mientras estiman de manera precisa la posición del robot, incluso sin información externa como GPS.

- Aunque la odometría basada en LIDAR proporciona estimaciones precisas de la pose del robot, existen desafíos como la interferencia de superficies reflectantes, la variabilidad de los escaneos debido a la velocidad del robot y las condiciones del entorno, como la luz solar intensa o la lluvia. Estos factores deben ser cuidadosamente gestionados para mantener la precisión del sistema de SLAM.
- La construcción de mapas mediante LIDAR y la localización basada en landmarks identificados en los escaneos son procesos clave en la navegación autónoma. Técnicas como el algoritmo de *Iterated Closest Point* (ICP) son esenciales para mejorar la precisión de la localización y resolver el problema de la asociación de datos, especialmente cuando los puntos de referencia no son fácilmente identificables en entornos con estructuras simples o repetitivas.

## Referencias

- Chap6AdvanceNotes.docx
- Corke, P. *Robotics, Vision and Control (RVC)*
- Cuaderno de código: `chap6.ipynb`
- Documentación de Robotics Toolbox
- Repositorios código fuente. (`rvc` `robotic-toolbox`)